

Chanting Book Prompts — Batch

The same two-stage workflow rebuilt for volume: several chants per message, across a 286-chant book.

How to use this sheet

1. Open Claude.ai	Paste Stage 1 once at the start of a session. It sets the rules for the whole conversation.
2. Choose a depth	Put FULL, COMPACT or DATA-ONLY on the first line of your paste. See the table below. If you say nothing it uses COMPACT.
3. Paste a batch	Several chants at once, numbered. The reply lists every one it received before it writes any of them.
4. Count the manifest	Check the manifest against how many entries actually arrived. If they differ, the reply was cut off — re-paste from the id in resume_from. This is the check batching exists to need.
5. Check against the book	The closing sentence counts what needs verifying. Do that with the book open. This is the step nothing else can do for you.
6. Open Claude Code	Paste Stage 2, then the whole reply underneath it. It reconciles the manifest before it writes anything.

Choose a depth before you start

A finished entry is about nine times the size of what you paste in, and Thai script, IAST and Paiboon+ all tokenise about twice as expensively as English. That arithmetic — not the wording of the prompt — is what limits how many chants fit in one reply. So the batch sheet lets you trade commentary for chants.

FULL	Everything the one-chant sheet produces. 2–3 chants per reply.
COMPACT	background 1 paragraph, meaning 2. Everything else in full. 4–6 chants per reply.
DATA-ONLY	Book content and structure only — no background, meaning, summary, when_chanted, source or per-verse english. 8–12 chants per reply.
Why it works	About 58% of a finished entry is commentary written about the chant, and none of it needs checking against the physical book. DATA-ONLY defers that half and captures the half that needs the book open.

The one rule that matters most

Neither stage may ever reconstruct Pali or Thai from memory. A model asked to fill a gap will produce something that reads beautifully and is not what your book says — and nothing about it will look wrong. Both prompts are built to raise a flag instead of filling a gap. Every flag it raises is a line for you to check against the physical book.

Paiboon+ quick reference

This app uses Paiboon+, not RTGS. The difference that goes wrong most often is aspiration: in Paiboon+ a bare k already means the aspirated sound, because g is reserved for the unaspirated ṇ. Never write kh, th or ph.

Never write ng — ง is always ฦ.

Consonants

ก = g ข ค ฦ = k ง = ฦ จ = j ฉ ช ฦ = ch
ด = d ต = dt ถ ท ฦ = t บ = b ป = bp
ผ พ ภา = p ฟ ฟ = f ข ศ ษ ส = s ห = h ร = r ล = l

Vowels

ε (แ) ɔ (อ) ə (เออ) ʉ (อี)

Tones

mid unmarked low à falling â high á rising ắ

Watch for

พระ → prá not phrá • หัง → táɲ not thâng
ขันธุ์ → kǎn not khǎn • ของ → kǔwɲ not khǎaw

Stage 1 — paste into Claude.ai

Paste this once at the start of a session, then paste batches of chants. It returns one JSON object holding a manifest, an array of entries, and a status.

You are helping compile a Theravāda chanting book. I am the editor; you are the typist and translator. I will paste text I have copied from an official Thai chanting book — SEVERAL chants at a time — and you will return them in a fixed five-layer format, as one JSON object containing an array.

Batch mode — read this section twice

I am working through a book of 286 chants. I will paste batches, numbered, with a depth on the first line. You return ONE JSON object per batch.

Two rules govern everything else in this section, and they outrank speed, tidiness and my stated chant count:

FINISH OR DON'T START. Never begin a chant you cannot complete. A missing chant is obvious to me; a truncated one is not, and a truncated one can reach the app looking finished.

THE MANIFEST DESCRIBES WHAT I GAVE YOU, NOT WHAT YOU MANAGED. Never trim it to match your output. Its whole purpose is to disagree with your output when something went wrong.

The manifest comes first

The FIRST key of your reply is `batch`. It names every chant I pasted, in the order I pasted them, before you write a single entry.

This ordering is deliberate, not stylistic. If your reply is cut off, the end of it is gone — so nothing written at the end can tell me it was cut off. A count written at the START survives, because I compare it against how many entries actually arrived. If the manifest says eight and six arrived, I know how many are missing, which ones, and that I should not trust the sixth.

So: manifest first, entries second, status last. If the status is missing, the reply truncated, and the manifest tells me where to resume.

Depth

I put one of these on the first line of my paste:

FULL — every field, as the single-chant prompt produces.
COMPACT — background 1 paragraph, meaning 2. Everything else in full.
DATA-ONLY — omit background, meaning, summary, when_chanted, source, and the per-verse "english". Keep pali, pali_roman, thai, paiboon, sections, titles, invitation and checks. This is the fidelity half of the job; the commentary comes later.

If I give no depth, use COMPACT. Never quietly change depth mid-batch. If you switch — because entries are running longer than expected — say so in `batch\_status.notes` and name the chant where you switched.

Repeats

In a 286-chant book the same chant appears in several different services. If a chant in this batch is one you have already set earlier in this session, do NOT set it a second time. Emit a stub instead and move on:

```
{"id": "<the id you used before>", "repeat_of": "<same id>",  
  "note": "<where it appeared before, and any difference in how it is printed>"}
```

If the two printings differ at all — a verse more, a different invitation, a different title — that is NOT a repeat. Set it in full and raise a check saying how it differs from the earlier one.

Check yourself before you close the batch

Before writing `batch\_status`, scan your own `paiboon` values for these four strings: "kh", "th", "ph", "ng". None of them can legitimately appear — Paiboon+ uses k, t, p and ṭ. Report the result in `batch\_status.paiboon\_scan` as a plain count of hits, and fix any you find before you send. This is a drift check: over a long session the pull back towards RTGS is strong, and it is the single failure this prompt most exists to prevent.

Also confirm in `batch\_status` that no `pali` value has been run through Paiboon+, which is the other corruption that cannot be spotted downstream.

The five layers — these are NOT interchangeable

A Thai chanting book prints two different things that both look like Thai script. Confusing them corrupts the whole entry, so read this twice:

1. pali — the PALI, written in THAI SCRIPT. This is what is actually chanted. It is Pali, not Thai. NEVER romanise this with Paiboon.
2. pali_roman — the SAME Pali in Latin script with Pali diacritics (IAST: ā ī ū ṃ ṇ ñ ṭ ḍ ṇ Ṃ). Romanised Pali, NOT Paiboon.
3. thai — the THAI TRANSLATION of that Pali. Ordinary Thai, never chanted, there for meaning.
4. paiboon — Paiboon+ romanisation of layer 3 ONLY. Never of layer 1.
5. english — the meaning in English.

Verse breakdown — make it a book people can read and chant from

Break it down verse by verse, exactly as the book does: ONE Pali line per verse, with its own Thai translation directly under it. A reader must be able to chant one line, read what it means, and move to the next — that is the whole test. Do not group several Pali lines together, and do not merge short lines to make them look even; a verse that is three words long stays three words long.

Where the source runs several Pali lines together before giving all their translations, pair them up by MEANING and keep the Pali in the order the book prints. Say in that chant's notes that you did this so I can check it.

If the chant has titled sections (movements such as "Reflection on the Body"), mark where each one starts. They are part of how the book reads.

Ground rules — the important ones

- Work ONLY from the text I paste. Never reconstruct, complete or "correct" Pali or Thai from your own memory of the canon. I verify every line against the physical book, and a plausible invention is worse to me than a gap.
- Batching changes nothing about this. Do not let a long batch tempt you into filling a gap to keep a rhythm going.
- If a line is unclear, garbled by OCR, or looks like it is missing a word, DO NOT guess. Reproduce what I gave you and raise a check.
- Reproduce layers 1 and 3 exactly as I paste them, character for character, including ṃ and any unusual spacing.
- If the source gives no Pali title, no invitation line, or no canonical source, leave that field empty and say so. Never fill a gap to look complete.
- You are writing layers 2, 4 and 5, plus the background and meaning sections.

Register — keep it, don't modernise it

The Thai in these books is formal, literary Buddhist Thai — roughly the Thai equivalent of the Book of Common Prayer. Mostly standard Central Thai grammar, but with scriptural vocabulary (บุคคล not คน), Pali loanwords (ขันธ, ตัณหา, นิพพาน), and older constructions (เบื้องบนแต่พื้นเท่านั้นมา rather than ตั้งแต่ฟ้าเท่านั้นมา). It is NOT royal language (ราชาศัพท์) and not a special monk language.

Never simplify or modernise the Thai. Match that register in English too: dignified and plain, closer to "Having gone to such a refuge, one is freed from all suffering" than to "Once you take this refuge you won't suffer".

Paiboon+ convention — follow this exactly

This app uses Paiboon+, a custom system derived from Benjawan Poomsan Becker's Paiboon. It is NOT RTGS. The distinction that goes wrong most often is aspiration:

ก = g	ข ค ฆ = k	ง = ŋ	จ = j	ฉ ช ฌ = ch
ด = d	ต = dt	ถ ท ฐ = t	บ = b	ป = bp
ผ พ ภ = p	ฝ ฟ = f	ข ศ ษ ส = s	ห = h	ร = r ล = l

So: พระ → prá (never "phrá"), หัง → táŋ (never "thâng"),
ขันธ → kǎn (never "khǎn").

Note: "k" in Paiboon+ already means the aspirated sound (ข ค), because "g" is reserved for the unaspirated ก. Never write "kh", "th" or "ph". Never write "ng" — ง is always ŋ.

Watch the vowels and finals as carefully as the consonants: ขลง is kǎwŋ (long วว, final ŋ). Writing it "khǎaw" would be a different word entirely.

Vowels use ะ ั ็ ุ ู ุ่ ุ้ (แ, อ, แอ, อี้). Tones: mid unmarked, low à, falling â, high á, rising ă. EVERY syllable carries its tone. Syllables joined by hyphens within a word, spaces between words, all lower case.

Working notes — one set per chant, inside that chant

Each entry carries its own `working\_notes`. Keep each note to a sentence or two:

1. ****invitation**** — present or absent. If absent, say so explicitly and say you have not written one.
2. ****units**** — how many Pali units you identified, and how you grouped them into the chant's natural movements.
3. ****ordering**** — anywhere the Pali and Thai do not run in step, or a translation appears before its Pali. Say you have kept my text as pasted and only noted the mismatch.
4. ****conventions**** — if anything I pasted uses a different romanisation from the Paiboon+ table above (RTGS spellings such as `sangkhaan`, `phró`, `thiàng`), say so and confirm you have used the table, not my sample. I sometimes paste output from other tools as a layout example; the layout is the example, never the romanisation.

Reproduce oddities, don't tidy them

If the source has a missing space, a doubled comma, an unusual spacing or an inconsistent particle, reproduce it exactly and raise a check saying what looks odd. Silent tidying is the failure this whole prompt exists to prevent.

Where the checks go

Checks are per chant, in that chant's own `checks` array — never in a shared top-level one, because across eight chants a pooled check cannot be traced back.

Every check names the verse it belongs to via its `verse` number, or `null` if it concerns the chant as a whole.

Raise a check for: anything OCR-garbled or possibly mistyped, a spelling that differs from standard editions, Pali and Thai running out of step, a missing invitation or title, an inferred line break, an attributed source, a chant that resembles an earlier one without matching it, and any typographic oddity you reproduced rather than tidied.

Ids

Propose the `id` yourself: the Pali title slugged — lower case, hyphens, no diacritics (`ariyadhana-gatha`, `mettanisamsa-sutta`). Where the book gives no Pali title, slug the English. The id appears in the manifest before the entry does, which is what makes the manifest checkable.

Output format — one JSON object

Your whole reply is ONE JSON object, then one closing sentence and nothing else. No prose before it, no markdown fences, no commentary except that one sentence. The object gets pasted straight into a tool that writes it into the app, so it has to parse.

Two formatting rules that matter more than they look:

- **Every value on ONE line.** Never wrap a long string across lines, and never put a literal newline inside a string. Thai script, Paiboon+ diacritics and IAST all survive copy-paste badly once a line wraps.
- **Manifest first, entries second, status last.**

```
{
  "batch": {
    "depth": "<FULL | COMPACT | DATA-ONLY>",
    "received": "<how many chants I pasted>",
    "manifest": ["<id of chant 1>", "<id of chant 2>", "<...every one I pasted>"]
  },
  "chants": [
    {
      "id": "<slug>",
      "working_notes": {
        "invitation": "<present, or absent and therefore not written>",
        "units": "<how many Pali units, and how you grouped them>",
        "ordering": "<any place the Pali and Thai do not run in step, or none>",
        "conventions": "<any romanisation conflict in what I pasted, or none>"
      },
      "title_thai": "<the chant's title in Thai script>",
      "title_pali": "<the title in IAST, or \"\" if the book gives none>",
      "title_roman": "<the THAI title romanised so a non-Thai reader can find the chant in a printed book. Only where the title is Thai; use \"\" where the title is Pali-in-Thai-script and title_pali already covers it.>",
      "title_english": "<the traditional English title>",
      "source": "<canonical source. If compiled from several places, start with \"Composite.\" and say which verses come from where. Use \"\" if you are not certain — do not guess. Omit at DATA-ONLY.>",
      "when_chanted": "<one sentence on when it is recited. Omit at DATA-ONLY.>",
      "summary": "<one sentence, max 30 words, for the index card. Omit at DATA-ONLY.>",
      "english_unverified": "<true if the book prints this chant in Pali only, with no Thai translation, so the English is a working translation made for this edition; otherwise omit the key>",
      "background": ["<paragraph>", "<paragraph>"],
      "meaning": ["<paragraph>", "<paragraph>", "<paragraph>"],
      "invitation": {
        "pali": "<Thai script, or \"\">",
        "pali_roman": "<IAST, or \"\">",
        "thai": "<the Thai translation of the invitation, or \"\">",
        "paiboon": "<Paiboon+ of that thai line, or \"\">"
      }
    }
  ]
}
```

```

    "english": "<meaning, or \"\">"
  },
  "verses": [
    {
      "number": 1,
      "section": "<สังขาร: The Three Characteristics — only on the FIRST verse of a section; omit the key otherwise>",
      "pali": "<ONE Pali line, Thai script>",
      "pali_roman": "<the same line, IAST>",
      "thai": "<its Thai translation>",
      "paiboon": "<Paiboon+ of the thai line only>",
      "english": "<its meaning. Omit at DATA-ONLY.>"
    }
  ],
  "checks": [
    { "verse": 6, "issue": "<what looks wrong and what to compare it against>" },
    { "verse": null, "issue": "<a check about the chant as a whole>" }
  ]
}
],
"batch_status": {
  "completed": ["<ids you finished in full>"],
  "not_started": ["<ids from the manifest you did not begin>"],
  "resume_from": "<the id I should re-paste from next, or \"\" if you finished them all>",
  "paiboon_scan": "<count of kh/th/ph/ng hits found in your own paiboon values, and that you fixed them — or 'clean'>",
  "pali_untouched": "<confirm no pali value was romanised with Paiboon+>",
  "notes": "<anything else: a depth switch mid-batch, a repeat you skipped, a chant that ran much longer than the rest>"
}
}

```

`background` and `meaning` are arrays — one string per paragraph, omitted entirely at DATA-ONLY. A chant's `checks` is empty (`[]`) only if genuinely nothing needs verifying, which is rare.

Sections — name them the way the book thinks

A `section` value carries BOTH names: the Thai or Pali name in Thai script, then the English. So:

```

"section": "สังขาร: The Three Characteristics"
"section": "มรณังคตัสสะติ: The Recollection of Death"
"section": "กายะคตาสสะติ: Reflection on the Body"

```

Only the FIRST verse of each section carries the key; leave it off the rest.

A long chant is several short reflections in sequence, and the Thai name is what a Thai practitioner would call that movement. If the book prints section headings, use those. If it does not, group by subject and say in that chant's working notes that the grouping is yours, not the book's.

Romanised Pali follows the Thai script beside it

`pali\_roman` must transliterate the `pali` line as printed, letter for letter. If that produces something a standard edition spells differently, keep the faithful transliteration and raise a check saying so — do not quietly substitute the standard form. The two chanted layers have to agree with each other, and a reader comparing them must not find them disagreeing without explanation.

The closing sentence

After the JSON, add nothing except one plain sentence giving the two numbers I need to decide what to do next — how many chants came back complete, and how many things need my eye:

"Six of the eight are complete; twenty-one things need your eye against the physical book; resume from pabbatopama-gatha."

Confirm you understand, then wait. I will paste the first batch.

Stage 2 — paste into Claude Code

Paste this, then the whole Stage 1 reply underneath it.

Add a BATCH of chants to the Digital Chanting Book in ~/thaibridge-ai.

The data lives in chanting.py, in the CHANTS list. Append one dict per chant, following the EXACT shape of the existing chants — same keys, same order, same comment style. Do not touch the route or the template; adding chants should need no other changes.

I'm pasting a JSON object from stage 1. It holds `batch` (a manifest), `chants` (an array of entries) and `batch\_status`. The trailing sentence after the closing brace is a note to me; ignore it.

Reconcile the manifest BEFORE you write anything

This is the first thing you do, and nothing else happens until it passes.

1. Count the ids in `batch.manifest`. Count the entries in `chants`.
2. If they do not match, DO NOT WRITE ANY OF THEM. Stop and tell me: how many were expected, how many arrived, which ids are missing, and what `batch\_status.resume\_from` says. A short batch means the reply was truncated, and the last entry that did arrive may be incomplete in ways that will not show up as a parse error.
3. If they match, check `batch\_status.completed` lists every manifest id too. Report any disagreement between the three before continuing.
4. Sanity-check the final entry specifically: does it have a `checks` array and a closing verse? A truncation that happens to land on valid JSON shows up here.

Say "manifest reconciled: N of N" and then proceed.

Ids and repeats — this book has 286 chants and they overlap

- Stage 1 proposes an `id`. Check every one against the ids already in CHANTS. On a collision, do NOT overwrite and do NOT silently suffix a number. Stop and show me both chants so I can decide whether it is a genuine repeat, a different printing of the same chant, or two chants that just slug alike.
- An entry with `repeat\_of` set is a chant stage 1 recognised as one already done. Do not add it. List it in your report so I can confirm the skip.
- If `title\_pali` is "", slug the id from the English or Thai title instead — never invent a title.

Mapping

Keys map straight across: title_thai, title_pali, title_roman, title_english, source, when_chanted, summary, english_unverified, background, meaning, invitation, verses. Each verse keeps its 'number', and its 'section' where it has one. Set 'group': 'General chanting' unless I say otherwise.

Two keys do NOT go in as data:

- working_notes → ignore. It is Josh's reading aid.
- checks → turn each entry into a !! code comment. A check with a "verse" number goes directly above that verse; a check with "verse": null goes at the top of that chant's dict. NEVER drop one silently — they are what Josh still has to verify against the physical book. Report the count PER CHANT, not as a batch total; a total hides a chant whose checks all vanished.

Depth — a DATA-ONLY batch is deliberately incomplete

Read ``batch.depth``.

- FULL / COMPACT — every field should be present. If one is missing, say so rather than writing the chant without it.
- DATA-ONLY — background, meaning, summary, when_chanted, source and the per-verse english are absent ON PURPOSE. Do NOT write them yourself, do not leave a placeholder, and do not flag them as errors. Write the chant without them and put one comment at the top of the dict:

```
# !! COMMENTARY PENDING: set DATA-ONLY. background, meaning, summary,  
# when_chanted, source and per-verse english still to be written.
```

The template guards each of those with ``{% if %}``, so the chant renders fine without them — it just reads as text only until the commentary is added. Grep for COMMENTARY PENDING to find them all later.

Structure rules

- ONE Pali line per verse, exactly as the JSON gives them. Never merge verses and never split one. Short verses stay short — a chant is read straight down the page, one line chanted then understood.
- 'section' marks where a new titled movement begins. Only the first verse of a section carries it; the template renders it as a heading above that verse.
- Empty strings are meaningful, not missing data. If every field of 'invitation' is "", keep the dict with its empty fields; the template checks it and skips it. If 'source' is "", leave it "". Do not fill any of them in to look complete.
- If you attribute a source yourself, mark it with a !! UNVERIFIED comment.

Fidelity rules — these matter more than tidiness

- Copy every Pali, Thai, Paiboon and English string across CHARACTER FOR CHARACTER. Do not fix, tidy, re-space or "improve" any of it. If something looks wrong, add it unchanged and tell me afterwards.
- Do NOT convert the Paiboon into another romanisation. This app uses Paiboon+: `η` never "ng", and `k/t/p` never "kh"/"th"/"ph".
- Watch the encoding: Thai script plus IAST diacritics plus Paiboon+ characters (`ε ɔ ə ɸ η`). Nothing should end up as ? or mojibake.

Then verify before telling me it's done

Run these with `PYTHONIOENCODING=utf-8` set, or Thai output will crash on Windows.

1. ``python -c "import chanting"`` — it must import cleanly.
2. Confirm `len(CHANTS)` grew by exactly the number of entries you added, and that every id is unique.
3. For EACH chant, compare the Thai and Pali you wrote against my pasted block and report character counts both ways. Per chant, not summed — a batch total can balance while two chants are individually wrong.
4. Re-scan the written file for "kh", "th", "ph" and "ng" inside paiboon values only, and report the count. Stage 1 checks itself; this checks stage 1.
5. Render the page and confirm each new chant shows the right verse count, its section headings and its layers, and that existing chants are unchanged.

Report as a table: id, verses in, verses written, checks carried, char match y/n. One row per chant. I read the table, not the prose.

Do not commit. I'll review the batch first.

Here is the batch:

[paste the whole stage 1 reply here — the JSON object and its closing sentence]

Before you say it is done

The canonical copy of both prompts is prompts/chanting-book-batch/prompt.md in the repo — copy from there, not from this sheet, which folds long lines to fit the page and prints !! where the prompt uses the warning emoji.

Count the manifest against the entries that actually arrived. A reply cut off mid-array still parses as JSON, so nothing downstream will notice on your behalf — and nothing written at the end of a reply can report it, because in a truncated reply the end is exactly what is missing. That is why the manifest comes first.

Check the character counts Stage 2 reports, both ways: every Thai and Pali character from your paste should appear in the file, and none should appear that was not in your paste. That the page loads and the file imports proves neither.